

Image recognition NN

Objectives:

- Create a Neural Network that will recognize images using:
 - Shallow Multi-Layer Perceptron (MLP)
 - Convolutional Neural Network (CNN)
 - Transformer (Encoder)
- Augment the images in test set in order to increase models' accuracy
- Attack the models using:
 - Fast Gradient Sign Method (FGSM)
 - Iterative FGSM (I-FGSM) / Projected Gradient Descent (PGD)
 - Momentum I-FGSM

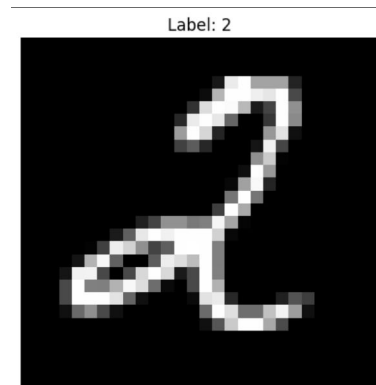
Input

MNIST - consists of **70,000 28x28 black-and-white images of handwritten digits extracted** from two NIST databases. There are 60,000 images in the training dataset and 10,000 images in the validation dataset, one class per digit so a total of 10 classes, with 7,000 images (6,000 train images and 1,000 test images) per class.

Split (2)	
train · 60k rows	
Search is not available for this dataset	
image	label
image · width (px)	class label
	
28 28	10 classes
	5 5
	0 0
	4 4
	1 1

Input data processing

1. Import and load using hugging face
2. Convert data into a correct format - tensor - normalize it to PyTorch format
3. Normalized images using MNIST statistics:
 - a. Mean = 0.1307
 - b. Standard Deviation = 0.3081
4. Extract image and label
5. Visualize in order to make sure it displays a correct image and label
6. Define the training and test dataset - 60k training, 10k test
 - a. Train ONLY on ds["train"]
 - b. Evaluate ONLY on ds["test"]



Multi-Layer Perceptron (MLP)

1. Simplest neural network architecture
2. Images flattened:
 - a. 28×28 to 784 features
3. Architecture:
 - a. Input Layer: 784 neurons
 - b. Hidden Layer: 128 neurons
 - c. Output Layer: 10 neurons
4. Advantages:
 - a. Simple implementation
 - b. Fast training
5. Limitation:
 - a. Loses spatial information in images
6. Result
 - a. Test Accuracy: ~93–95%

Convolutional Neural Network (CNN)

1. Uses convolutional filters to detect visual patterns
2. Preserves spatial relationships between pixels
3. Architecture:
 - a. Convolution Layer 1
 - i. Max Pooling
 - b. Convolution Layer 2
 - i. Max Pooling
 - c. Fully Connected Layers
4. Learns:
 - a. Edges
 - b. Curves
 - c. Digit shapes
5. Result
 - a. Test Accuracy: 98.80%

CNN with Data Augmentation

1. Added image transformations during training:
 - a. Random Rotation
 - b. Random Translation
2. Purpose:
 - a. Improve generalization
 - b. Reduce overfitting
 - c. Increase robustness
3. Result
 - a. Test Accuracy: 98.87%
4. Observation
 - a. Highest-performing model
 - b. Selected for adversarial attack testing

Transformer Encoder

1. Divides image into patches
2. Treats image as a sequence
3. Uses self-attention mechanism
4. Architecture:
 - a. Patch Embedding
 - b. Transformer Encoder Layers
 - c. Classification Layer
5. Result
 - a. Test Accuracy: 98.60%
6. Observation
 - a. Competitive performance
 - b. Slightly below CNN on MNIST
 - c. Transformers typically excel on larger datasets

Model comparison - outcomes

Model	Accuracy
MLP	93–95%
CNN	98.80%
CNN + Augmentation	98.87%
Transformer	98.60%

Adversarial Attacks

1. Goal:
 - a. Test robustness of the best model
2. Selected Model:
 - a. CNN + Augmentation
3. Baseline Recognition Rate:
 - a. 98.60%
4. Evaluated attacks:
 - a. FGSM
 - b. I-FGSM / PGD
 - c. Momentum I-FGSM

Fast Gradient Sign Method (FGSM)

1. Single-step adversarial attack
 - a. Uses gradient information to modify input image
2. Formula:
 - a. Perturbation added in direction that maximizes classification loss
3. Characteristics:
 - a. Fast
 - b. Simple
 - c. Less powerful than iterative attacks
4. Result
 - a. Attack Success Rate (ASR): 6.88%

I-FGSM / PGD Attack

1. Iterative version of FGSM
2. Performs multiple attack steps
3. Recalculates gradients after each step
4. Stronger optimization process
5. Result
 - a. Attack Success Rate (ASR): 10.27%
6. Observation
 - a. Strongest attack in experiment

Momentum I-FGSM Attack

1. Extends iterative attack with momentum
2. Accumulates gradient directions
3. Helps avoid oscillation
4. Often improves attack transferability
5. Result
 - a. Attack Success Rate (ASR): 9.98%

Results

Method	Result
Recognition Rate (Clean Data)	98.60%
FGSM	6.88% ASR
I-FGSM / PGD	10.27% ASR
Momentum I-FGSM	9.98% ASR

Conclusions

1. CNN-based models significantly outperformed MLP
2. CNN + Augmentation achieved the highest classification accuracy
3. Transformer performed competitively but did not surpass CNN
4. Adversarial attacks successfully fooled the model despite high accuracy
5. Iterative attacks (PGD and Momentum I-FGSM) were more effective than FGSM
6. Results demonstrate that:
7. High accuracy does not guarantee robustness
8. Neural networks remain vulnerable to adversarial examples